

DSA & Competitive Programming

Complete Beginner Roadmap — Java Edition

This roadmap takes you from zero to competitive programmer step by step. Each phase builds on the last. Don't skip — logic building requires a strong foundation. All code examples are in Java.

Phase 1 — Java Fundamentals (Weeks 1–3)

What to learn:

- Variables, data types, and operators
- Conditionals (if/else, switch)
- Loops (for, while, do-while)
- Functions / methods
- Arrays and Strings
- Basic input/output with Scanner

Java Example — Basic I/O & Loops:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        // Print sum of 1 to n
        int sum = 0;
        for (int i = 1; i <= n; i++) {
            sum += i;
        }
        System.out.println("Sum = " + sum);
    }
}
```

Practice Goal: Solve 30 easy problems on HackerRank (Java domain).

Phase 2 — Recursion & Basic Math (Weeks 4–6)

What to learn:

- Recursion — how the call stack works
- Factorial, Fibonacci, power functions
- GCD / LCM using Euclid's algorithm
- Prime numbers — Sieve of Eratosthenes
- Modular arithmetic ($a \% MOD$)

Java Example — Recursion (Fibonacci):

```
public static int fib(int n) {
    if (n <= 1) return n; // base case
    return fib(n - 1) + fib(n - 2); // recursive case
}

// Note: Use memoization for large n (see Phase 4)
```

Java Example — Sieve of Eratosthenes:

```
boolean[] isPrime = new boolean[n + 1];
Arrays.fill(isPrime, true);
isPrime[0] = isPrime[1] = false;

for (int i = 2; i * i <= n; i++) {
    if (isPrime[i]) {
        for (int j = i * i; j <= n; j += i)
            isPrime[j] = false;
    }
}
```

Practice Goal: 20 recursion problems on LeetCode (Easy).

Phase 3 — Core Data Structures (Weeks 7–11)

What to learn:

Data Structure	Java Class	Key Operations
Array / ArrayList	ArrayList<Integer>	get, add, remove — $O(1)/O(n)$
Stack	Stack<Integer>	push, pop, peek — $O(1)$
Queue	LinkedList / ArrayDeque	offer, poll, peek — $O(1)$
HashMap	HashMap<K,V>	put, get, containsKey — $O(1)$
HashSet	HashSet<Integer>	add, contains, remove — $O(1)$
Linked List	LinkedList<Integer>	addFirst, addLast — $O(1)$

Java Example — Stack for Balanced Parentheses:

```
public static boolean isBalanced(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push(c);
        else if (c == ')') {
            if (stack.isEmpty()) return false;
            stack.pop();
        }
    }
    return stack.isEmpty();
}
```

Practice Goal: 40 problems mixing arrays, stacks, queues on LeetCode.

Phase 4 — Algorithms (Weeks 12–18)

What to learn:

- Sorting — Bubble, Merge, Quick Sort
- Searching — Binary Search (very important!)
- Two Pointers and Sliding Window
- Dynamic Programming — Memoization & Tabulation
- Greedy Algorithms
- Graph basics — BFS and DFS

Java Example — Binary Search:

```
public static int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1; // not found
}
```

Java Example — DP (0/1 Knapsack):

```
// dp[i][w] = max value using first i items with capacity w
int[][] dp = new int[n + 1][W + 1];
for (int i = 1; i <= n; i++) {
    for (int w = 0; w <= W; w++) {
        dp[i][w] = dp[i-1][w]; // skip item
        if (weight[i] <= w)
            dp[i][w] = Math.max(dp[i][w],
                                dp[i-1][w - weight[i]] + value[i]);
    }
}
```

Practice Goal: Solve 50+ medium problems. Start Codeforces Div. 3 contests.

Phase 5 — Competitive Programming (Weeks 19+)

What to learn:

- Advanced Trees — BST, Segment Tree, Fenwick Tree
- Advanced Graphs — Dijkstra, Floyd-Warshall, MST
- String algorithms — KMP, Z-algorithm
- Bit manipulation tricks
- Number theory — Euler's totient, fast exponentiation

Competitive Programming Platforms:

Platform	Best For	Start With
LeetCode	DSA practice, interviews	Easy → Medium
Codeforces	CP contests	Div. 3 / Div. 4
HackerRank	Language basics	Java track
CSES Problem Set	Classic CP problems	Sorting section
AtCoder	Clean problem quality	ABC contests

Suggested Daily Routine

Time	Activity
30 min	Review yesterday's concept / notes
60 min	Solve 1–2 problems (don't give up before 30 min)
30 min	Read editorial / watch solution if stuck
20 min	Re-implement the solution from scratch

Key Rule: Consistency beats intensity. 1 hour every day is better than 7 hours once a week.